

Ambient Awareness of Build Status in Collocated Software Teams

John Downs

*Department of Computing and
Information Systems
The University of Melbourne
Melbourne, Australia
jdowns@student.unimelb.edu.au*

Beryl Plimmer

*Department of Computer Science
The University of Auckland
Auckland, New Zealand
beryl@cs.auckland.ac.nz*

John G. Hosking

*ANU College of Engineering and
Computer Science
Australian National University
Canberra, Australia
john.hosking@anu.edu.au*

Abstract—We describe the evaluation of a build awareness system that assists agile software development teams to understand current build status and who is responsible for any build breakages. The system uses ambient awareness technologies, providing a separate, easily perceived communication channel distinct from standard team workflow. Multiple system configurations and behaviours were evaluated. An evaluation of the system showed that, while there was no significant change in the proportion of build breakages, the overall number of builds increased substantially, and the duration of broken builds decreased. Team members also reported an increased sense of awareness of, and responsibility for, broken builds and some noted the system dramatically changed their perception of the build process making them more cognisant of broken builds.

Keywords—software teams; continuous integration; ambient awareness; build processes; status information

I. INTRODUCTION

The process of developing commercial software within a team is time-consuming and complicated. Many subtle problems can occur due to miscommunication, human error, misunderstanding of requirements, and myriad other reasons. The sooner these problems can be detected the easier they are to fix, and the lower the cost of fixing them [1]. A build process provides regular checkpoints where a system can be evaluated against objective criteria for quality, stability, and performance and where some of these problems may be detected. Build processes are increasingly complex and increasingly automated. Due to this automation they can be run very frequently, leading to continuous integration [2], which results in frequent sampling of the status and health of a development project. Once a build has occurred, team members must be made aware of the outcome of the build and given the opportunity to correct issues that may have been found [3].

However, notification competes with other activities developers are participating in and hence delays in resolving these issues frequently occur. In this paper, we describe the use of ambient devices to assist users to be more aware of build status. We begin by motivating our work from a previous study we have undertaken as well as related work. We then describe our ambient awareness based approach, its design and implementation. Results of a study of the system

in use in an industrial context are presented, followed by discussion, conclusions and future work.

II. BACKGROUND AND MOTIVATION

Due to the frequency of builds and volume of data generated by the build process, development teams need good ways to monitor build status and become proactively aware of issues that may be uncovered during builds. In a global software team study, [4] found that a broken build could be traced to communication breakdowns between team members. In an industrial study of an agile team [5], we found that, while developers considered the build process to be highly important in their daily workflow, there was a striking inconsistency between the team's preference that broken builds be avoided and a relatively high frequency of broken builds. Builds have a direct impact on the ability of the team to work on and complete the set of issues they have been assigned. We found that broken builds, particularly broken nightly builds, can add distractions and delays to this process, and are often avoidable.

Build monitoring systems can include automated emails at the conclusion of every build, publishing results to Twitter or an RSS feed, or even just the provision of a web page for team members to view build results on demand. These typically, however, compete for attention with other information: in our study, system-generated build status emails were regarded as being of low priority and hence filtered out, meaning knowledge of and responsibility for correcting build breakages was only slowly realized. Although a central build status screen provided aggregate status information it also failed to compete with more immediate stimuli. This is consistent with [6] who found strong commitment was needed by team members to ensure communication channels were used effectively.

Much work has focused on promoting awareness of project and code status to developers. Biehl et al. [7] describe FASTDash, a system to passively aggregate and display information about developers' activities to each other on a large screen in their work environment. Similarly, Palantir [8] provides developers with information about the types of code changes being made by others in real time, and presents this information in the developers' IDE. Both systems were predicated on the assumption that increasing awareness of other team members' activities would improve the process and output of software development.

Ambient devices present dynamic information in an at-a-glance manner and have low attentional requirements. Therefore they are appealing to solve the build notification problem observed in our study. Several approaches have attempted this. A lava lamp is used by [9]: when a build breaks, the lamp is turned on. This is argued to provide practical and social cues to correct problems rapidly. Others include use of the Ambient Orb [10], the Build Wallboard [11], the Nabaztag wireless rabbit [12], [13], LED signs [14], and a USB snowman [15].

However, while these systems are anecdotally appealing, there has been little formal assessment of their value. [16] performed a preliminary, small-scale, qualitative study of lava lamps and toy robots as build notification devices, but only evaluated team members' high-level impressions and did not examine the effects that these devices had on the team or on build quality. The paucity of research in this area motivated the work we present here.

III. OUR APPROACH

From our motivating study [5], we derived a set of high-level requirements for the construction of systems to monitor builds, improve build quality, and reduce broken builds. These are shown in Table 1.

TABLE 1. REQUIREMENTS FOR BUILD MONITORING SYSTEMS

R1	<i>Promote collective responsibility:</i> by making the entire team aware of broken builds or other problems, social pressure will be applied to resolve them.
R2	<i>Promote individual responsibility:</i> highlight when a developer may have broken a build. This must be balanced by the fact that broken builds are not necessarily a problem if resolved quickly, and that overtly blaming a developer may be counterproductive and engender resentment. The way in which information is presented has a substantial effect on this perception.
R3	<i>Passive collection:</i> wherever possible, systems should automatically collect information and make inferences. The overhead involved in having human involvement in collecting and disseminating build information is too great.
R4	<i>Timely:</i> systems should be dynamic and promptly updated, allowing developers the greatest chance of fixing the build successfully in the shortest amount of time, and lessening the scope for other confounding changes to occur while the build is in a broken state.
R5	<i>Separate channel of information:</i> existing channels such as email and IM have acquired meanings not conducive to dynamic build status information. If information is conveyed through a novel and distinct modality, it can draw the user's attention when appropriate and be easily ignored at other times.
R6	<i>Simplicity and ease of use:</i> information conveyed should be perceivable with little cognitive and physical effort. The system should avoid requiring developers to open web pages, interpret complex charts, find the most recent result in a list, or make calculations or inferences based on a set of data. This is particularly important since the system will be used multiple times every day.

Our proposed approach addresses these requirements by using ambient device technology. As noted above, many types of ambient device technology exist, but their fundamental tenets, constant availability of digital information within a physical environment, dynamic updating, ease of perception, and a separate channel to the user's standard workflow, provide a solid base on which to construct a build notification system.

A. Key Design Issues

Two key design issues relate to the role of ambient devices. First, it is important to assess the complexity of the information to be presented, and consider whether this information could become too cognitively intensive to perceive in a single glance. Second, consideration needs to be given to whether the devices should generate notifications only for exceptional cases – broken builds – or for all builds.

'Glanceability' dictates that the information an ambient device presents should be available effortlessly, with minimal cognitive processing. Some ambient devices have multiple dimensions along which information can be presented, allowing one device to convey two or more independent channels of information. However, taking advantage of these multiple channels requires additional cognitive effort to parse and becomes a confounding factor in any evaluation of the device. We decided to avoid these issues by restricting a given device to display one channel of information. If a device supported multiple channels, we chose to redundantly encode information, thus providing multiple cues that could be perceived independently but which carried the same information.

The second issue was whether to only notify developers of broken builds, or to also notify them of successful builds. Several factors favoured the former. First, an additional notification level would require more complex system behaviour, thus increasing the cognitive effort to interpret meaning. Second, if developers are notified their own code has successfully built, they may become less interested in the overall state of the build system. This is less likely if developers receive just failure notifications. Third, notifications after every commit could become distracting and intrusive thereby reducing the 'ambience'. Last, high notification volumes may diminish their value over time. If notifications only occur when builds fail they are less common, and may therefore be perceived as exceptional cases requiring urgent attention and action.

However, there are also arguments for the latter, based on the principles of positive reinforcement and positive punishment, specifically the Law of Effect [17]. If developers are only notified if a build fails, this could be construed as a form of positive punishment where a stimulus is delivered to reduce incidence of a behaviour. By the same logic, if developers are also notified if a build succeeds, they receive positive reinforcement. If the Law of Effect applies, it suggests reinforcers and punishers have substantial effect on behaviour, and that their combination would be stronger than only using punishers.

We chose to develop multiple prototype systems in order to compare the two distinct behaviours – build failure notifications only, and reinforcement and punishment-based notifications for all build results. This also permitted a comparison between the two conditions against each other and against a baseline condition where no devices were used.

B. Candidate Technologies

As noted earlier, a variety of ambient devices are available and have been applied to the build notification problem. We evaluated several of these, together with the

existing email and dedicated build status monitor used by our study participants, against our derived requirements. Several representative examples are shown in Table 2 (open circles represent a partially fulfilled requirement).

TABLE 2. CANDIDATE TECHNOLOGIES MATCHED AGAINST REQUIREMENTS

Technology	R1	R2	R3	R4	R5	R6
Email (existing system)	○	●	○	●	●	●
Dedicated build status monitor (existing system)	●		●			
Software running on developers' workstations		●	●	○		●
Large display	●		●	●	●	●
Ambient Orb	●		●	○	●	●
Personal LED		○	●	●	●	●
Nabaztag		●	●	●	●	●
Central USB light	●		●	●	●	●

A simplistic solution would involve installing software on developers' workstations. This would monitor the build server and provide visual notifications when a build had failed. However, such a solution would fail to meet a number of the requirements listed above, particularly the lack of separation from other workflow. Looking at the other ambient devices, we see that none cover all requirements; they tend to either focus locally, such as the Nabaztag and personal LED, or globally, such as a Central USB light. Accordingly, we chose a combination of two devices: a single shared central Delcom USB light (Fig. 1 left), for global notification, and individual Nabaztag devices (Fig. 1 right) for each developer, indicating they could be the source of a build failure.



Figure 1. Delcom USB light (left) and Nabaztag Wi-Fi rabbit (right).

C. Testable Hypotheses

Given the lack of data on ambient notification system efficacy, we were interested in measuring what impact such a system might have on the team's behaviour. To address this we have developed a set of testable hypotheses, shown in Table 3, which span the range of plausible impacts that might be expected from such a system, with arguments for each hypothesis.

TABLE 3. TESTABLE HYPOTHESES

	Hypothesis	Discussion
H1	The proportion of builds which fail will decrease.	We expect developers will aim to reduce the frequency of broken builds to avoid receiving build failure notifications. We particularly expect this when both positive reinforcement and positive punishment are provided.
H2	The overall number of builds per day will increase.	We expect developers to commit code more regularly, thus increasing build frequency. This is based on the expectation that developers will check in more granular changes to keep changes manageable, reducing the risk of broken builds.
H3	The number of files checked in per changeset will decrease.	If developers check in more regularly, we expect that the number of files per changeset will decrease as developers check in more granular changes.
H4	The number of commits to a broken build will decrease.	Developers commit to a broken build to fix the underlying build break problem, or to commit unrelated changes (perhaps unaware that the build is breaking at the time of their commit). As build status is easier to ascertain with the monitoring technology, we expect developers to be less likely to commit to a broken build except to fix it.
H5	The proportion of builds triggered by a manual request from a developer will increase.	We predict developers will seek to remove build failure notifications from their ambient devices rapidly. They will prefer to manually invoke a build once they have committed code to fix a problem, validating the change they have made, rather than waiting for the build server to automatically perform a build.
H6	The proportion of broken builds due to unit test failures will decrease.	Broken builds can occur for many reasons. The three primary reasons are unit test failures, compilation failures, and intermittent problems with infrastructure. Unit test failures are usually avoidable if a developer runs the unit test suite on their development machine before checking in. We predict developers will do this more often once the awareness technology is added, decreasing the proportion of broken builds due to unit test failures ^a .
H7	The proportion of failing builds fixed by the breaker will increase.	We expect developers' personal responsibility for fixing builds will increase as they are identified more quickly and publicly. Thus, we expect they will be more likely to fix broken builds they have committed to personally.
H8	The average duration of broken builds will decrease.	We expect the amount of time that builds remain in a broken state will decrease: developers will quicker address the problem with a persistent and prominent visual cue.
H9	The proportion of time in which any build plan is failing will decrease.	Compared to a baseline condition without any ambient device technology, we expect the total amount of time any build plan is in a failing state to decrease, as broken build duration decreases and frequency of overall builds increases.

a. Unfortunately, compilation failures and unit test failures are not distinguishable in the build server logs so we cannot test hypotheses about the nature of build failures.

IV. IMPLEMENTATION

Fig. 2 shows the architecture of the notification system developed and installed in the company participating in this study. The build server, an Atlassian Bamboo server [18], manages build plans, corresponding to a tree in the SVN repository built against a defined schedule. A REST-based API is used to obtain build results, which are processed by a custom Build Monitor component. This logs information about builds and their results and notifies a Device Server component about status changes. The Device Server, together with a jNabServer [19] component, manages interaction with the ambient devices. These components were modified to enable different notification behaviours for each of the prototypes we developed.

Three prototypes were developed. The first notifies individual developers, via their Nabaztags, when a plan they have just committed to has broken, and also notifies all developers via the centrally located USB light. The Nabaztag notification turns the rabbit's ears down and its LEDs red when a build breaks. When a broken build is fixed the ears are turned up and LEDs green. Between times ears are up with no LEDs on.

The second prototype extends this to notify all contributors to a *sequence* of broken builds, not just those contributing to the current build. The third prototype also provides positive reinforcement, notifying contributors to a build whenever it succeeds, not just when a broken build is fixed. In addition, a baseline system was implemented that omits the device management components and just monitors and logs build status.

Considerable effort went in to ensuring the prototypes could run for extended periods without manual intervention, including: robust error handling; management of potentially unreliable connections, such as the wireless link between jNabServer and the Nabaztag devices; and auto-restart code for system failures.

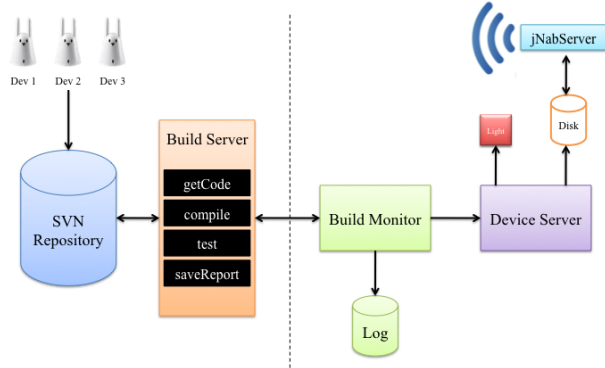


Figure 2. System architecture.

V. EVALUATION

A. Evaluation Approach

Evaluation of the system was via a case study. The system was installed in an industrial setting and used by a team of 11 agile developers as part of their normal work over a period of 3 months. 24 workdays were spent on baseline measurements, then 10 on prototype 1, 20 on prototype 2 and

16 on prototype 3, making 4 experimental conditions in total. In addition to gathering extensive data on builds from the system log, team members were interviewed at the end of the project to obtain qualitative feedback on their experiences.

When analysing the build data, several build types were excluded to avoid bias of the data. These were: nightly builds, which were not tightly coupled to developer workflow; failed builds due to unit test communication failures; and builds deleted from Bamboo before being logged, such as ones cancelled while in progress. From a total of 880 builds, 783 were analysed after these exclusions.

Given the unequal durations of the 4 experimental conditions (simply due to accidental circumstances), the metrics presented are typically percentages and averages to allow fair comparison between conditions.

For the metrics described in this section, no inferential statistics (such as chi squares, t tests, or analyses of variance) have been performed. This is deliberate, based on the best practices for the design of case studies and other small sample studies. The small sample size of a case study precludes inferential analysis, and hence does not permit any definitive conclusions about a population to be drawn from the sample. A given observation within a single sample could be a real effect, or it could be random variation due to other factors [20]. Case studies and other small-sample designs do not provide a sufficient quantity of data to allow these possibilities to be distinguished. Rather than an indictment upon case study methodologies in general, this is simply a feature of these types of study, and alternatives to inferential testing are available for the analysis of quantitative case study data, such as those we use here: baseline-intervention designs and descriptive statistics [21].

B. Results

This section outlines metrics calculated to test each of our candidate hypotheses, plus a description of the measurement process for each metric and the actual results. Charts with means and proportions are provided and, where applicable, standard deviations presented as error bars.

Fig. 3 shows the mean percentage of broken builds per workday for each condition. This shows a small decrease from the baseline when prototype 1 was installed but an increase beyond baseline for prototypes 2 and 3. By contrast the mean total number of builds (pass and fail) per day (Fig. 4) increased substantially over baseline for all prototypes.

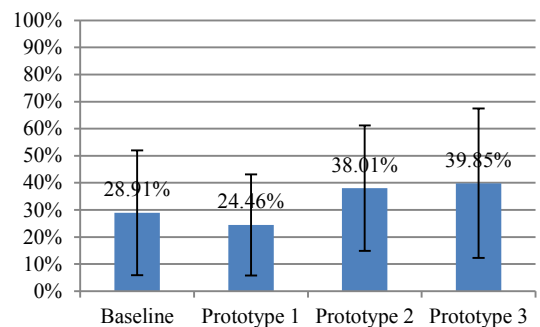


Figure 3. Mean percentage of broken builds per workday per condition.

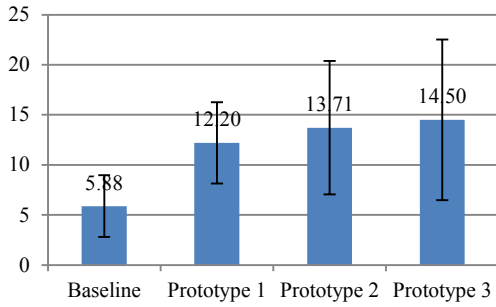


Figure 4. Mean total builds per workday.

Each build has an associated changeset, comprising the commits made by each developer since the previous build and with each commit comprising a set of files that have been changed. Fig. 5 shows the average total number of committed files per changeset for each condition. This shows a small rise over the baseline for each condition.

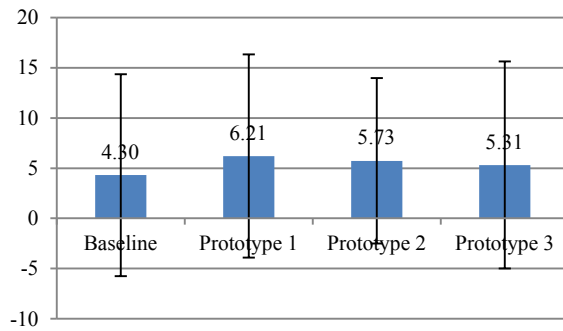


Figure 5. Mean files per changeset.

When a build breaks it may take several more failed builds before the build passes again. This sequence of failed builds is a *build chain*. Fig. 6 shows the mean number of commits to fix a broken build, i.e. the number of commits to terminate the broken build chain. This shows a small increase over the duration of the study.

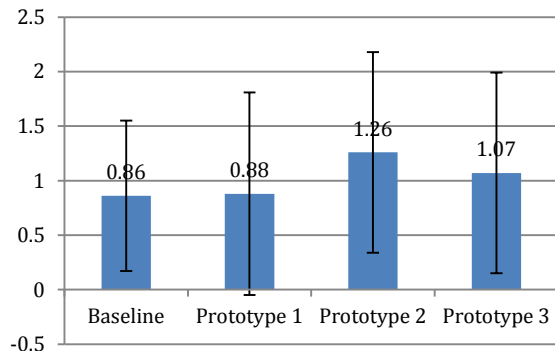


Figure 6. Mean commits to broken build chains.

Builds can occur for a variety of reasons: an initial build for a new or modified plan; after source code changes, with a timeout to gather multiple commits; or manually, when invoked directly by a developer who does not want to wait

for the next automatic build. Fig. 7 shows the variation of build trigger reason by condition. This shows a small increase over baseline for each condition.

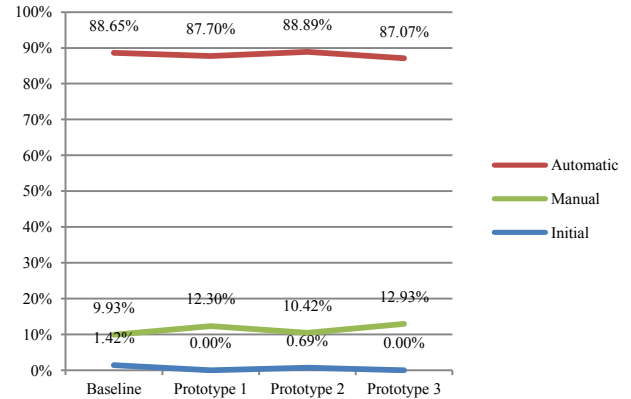


Figure 7. Percentage of builds by trigger reason.

The build server was only able to provide limited data on why a particular build failed. The only reason that could reliably be derived was ‘test failure’, indicating one or more unit tests had failed. Other problems included compilation errors, network outages, and infrastructure issues. These causes could not be distinguished, so are grouped together as ‘other’ causes of build failures. Fig. 8 shows the percentage of broken builds caused by unit test failures. This shows a decrease over the baseline for all prototypes.

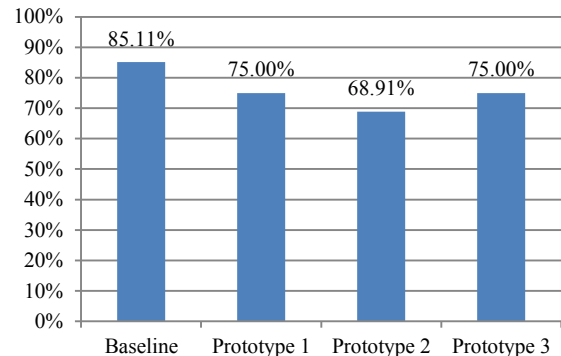


Figure 8. Percentage of broken builds caused by unit test failures.

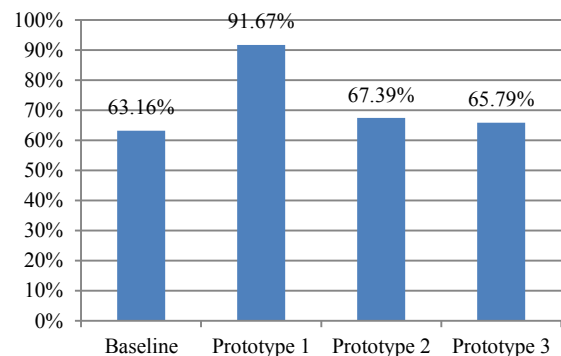


Figure 9. Percentage of broken build chains fixed by breaker.

We wanted to understand whether developers were fixing broken builds they had committed to. We calculated a list of those committing to a broken build and compared that with those committing to the next passing build. If the next passing build had one of the original committers it was considered to have been fixed by a breaker. This assumes that a developer who committed to a broken build was likely to be responsible for the breakage. Fig. 9 shows the percentage of broken build chains fixed by a breaker. This shows a strong increase for prototype 1 but falls back to close to the baseline thereafter.

The duration of a broken build is defined as the time between when the first build in a broken build chain ends to when the next passing build begins. For chains longer than one build, this also includes the build time for the intermediate failing builds. Some broken build chains were found to span two days so we have separate analyses for these, with an estimate of non-working time subtracted off their timings. Fig. 10 shows these two analyses. In both cases a reduction from the baseline was observed. In the case of the same day fixes, this showed a small drift back towards the baseline as the study progressed, while for the multi-day fixes, it was consistently lower.

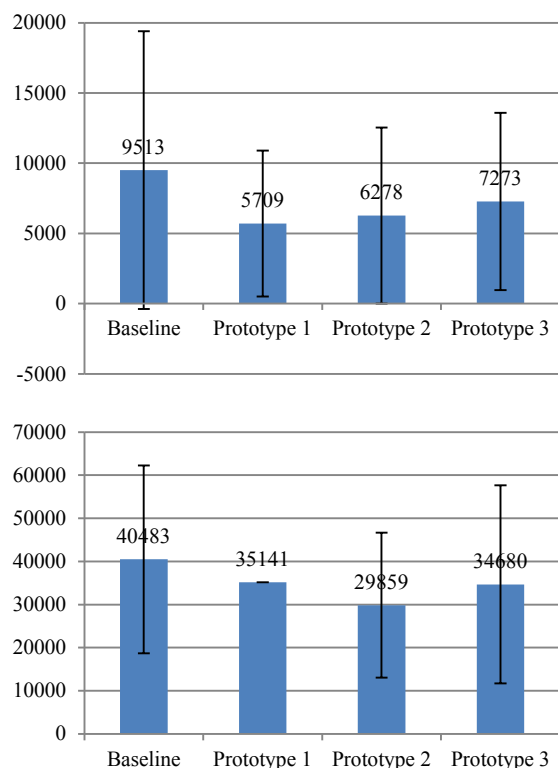


Figure 10. Mean fix duration (s) for builds fixed the day they were broken (top) and those fixed on a different day (bottom).

A final comparison was of the total amount of time the central light was red (indicating one or more build plans was failing) with the total amount of time that it was green (indicating all integration plans were passing). This metric was calculated for all conditions, including the baseline condition. Even though the central light was not provided to

the team during the baseline condition, the time the light would have displayed each colour if it had been present was calculated, and this information is included for comparison. As in the previous metric, this was calculated based on work hours of workdays. Fig. 11 shows the proportion of work time the central light showed red. This shows a strong dip with the introduction of prototype 1 but a steep rise above the baseline thereafter.

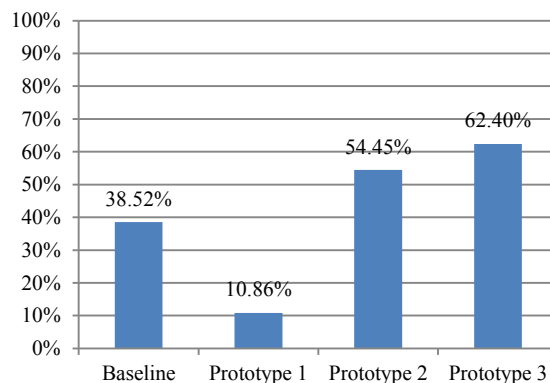


Figure 11. Percentage of work time the central light showed red.

C. Qualitative Feedback

One source of qualitative feedback was the level of engagement developers exhibited. During our visit to install the devices we observed some interesting behaviour from team members. Even before the devices were installed some team members began referring to broken builds as “sad bunnies”. Immediately after the system was installed a build failed, and one team member commented:

This is the first time I've noticed a broken build!

In the days following, team members engaged strongly, making suggestions and giving feedback. In particular, one developer – who had previously been considered (by himself and others) as having the least interest in broken builds – was the most active in making comments and suggestions. This feedback contributed to the design and development of prototypes 2 and 3.

Approximately one week after initial installation, the team requested an extra Nabaztag be set up as a surrogate for a developer who worked remotely. We considered this an indication of the team's acceptance of the technology, and a validation that team members found the system useful. The developer seated next to the surrogate rabbit reported he would send instant messages to the remote worker whenever the surrogate rabbit indicated a build failure.

Interviews were held with seven of the developers shortly after introduction of prototype 3 (time constraints prevented later interviews) and the results analysed for themes and informally clustered.

Overall, every developer was positive about the ambient devices, one commenting:

I think it solved pretty much all the problems with the old email system. I liked knowing quickly when I'd broken the build, I liked being able to point the finger when someone else broke the build.

Team members were unanimously positive about the Nabaztag devices, particularly the ear motion indicating a build break and about the accuracy of the candidate developers causing the break. The rabbit form factor was regarded positively, with some reporting that a quick scan around the room gave a good indication of overall build status and hence when it was safe to commit. The rabbits were regarded as fun and less threatening than email, the central light or build monitors.

The central light had more mixed reception with comments that it replicated the central status monitor or was in an inconvenient location. However, many noted they used it when walking in and out of the room to assess status.

On installation, some members questioned the need for a separate channel rather than using their own computer screens. However, this perception changed, with developers regarding the separate channel positively, one noting:

You get email from all sorts of things throughout the day, whereas this is a unique thing... you know straight away [what it means].

Members were asked to predict effects that might be observed from the build log analysis. Most commonly mentioned was the increased level of awareness and responsibility they had over build status followed by a reduced need for team members to “nag” each other and a heightened sense of priority to fix a broken build. All but one of those interviewed felt the time to fix a build had reduced.

Team members also suggested improvements, most frequently an indication of when a build was in progress to minimize the likelihood of checking in code during a build.

VI. DISCUSSION

In general, the introduction of the build monitoring system did have an effect on the team’s work and on their workflow for building software. Team members performed more builds, fixed broken builds more quickly, and may have been more productive as a consequence. Additionally, every team member interviewed was enthusiastic about the technology and expressed positive opinions. Based on these findings, we can reasonably state that the project was a success.

However, a number of fundamental questions remain: exactly what effects did the system have on the team, and why? Which components of the system worked well and which did not? Were the effects consistent or were they haphazard? Did they relate to individual developers’ behaviour, or the team’s behaviour as a whole?

A. Evaluation Against Hypotheses

Table 4 summarises the evidence of our hypotheses from the study data. As is true in many case studies involving the application of interventions, a Hawthorne effect can be observed. This effect is simply a response to the novelty of the introduction of the intervention, and will typically disappear within a relatively short period [22], [23]. In the case of this study, several of the metrics listed above demonstrated a characteristic Hawthorne pattern of an initial effect followed by a return to baseline levels. Four metrics in

particular – H1 (proportion of broken builds), H7 (builds fixed by the breaker), H8 (duration of broken builds), and H9 (time the central light showed red vs. green) – showed at least some degree of a noticeable behavioural shift in the early stages of the study that subsequently disappeared. These four metrics share a common theme in that they are all related to broken builds – namely, how often they occurred, how long they lasted, and who fixed them.

TABLE 4. LEVEL OF OBSERVED SUPPORT FOR HYPOTHESES

	Hypothesis	Evidence from Evaluation
H1	The proportion of builds which fail will decrease.	Ambiguous. The proportion of failing builds decreased when prototype 1 was installed, but then increased beyond baseline levels with prototypes 2 and 3.
H2	The overall number of builds per day will increase.	Supported. The mean number of builds per day increased each condition.
H3	The number of files checked in per changeset will decrease.	Not supported. The mean number of files per changeset increased each condition.
H4	The number of commits to a broken build will decrease.	Not supported. The mean number of commits per changeset increased over the study period relative to the baseline.
H5	The proportion of builds triggered by a manual request from a developer will increase.	Ambiguous. The proportion of manually triggered builds increased during prototype 1 relative to baseline, returned to baseline in prototype 2, and increased again in prototype 3.
H6	The proportion of broken builds due to unit test failures will decrease.	Supported. The proportion of builds breaking due to unit test failures decreased over the study period relative to the baseline.
H7	The proportion of failing builds fixed by the breaker will increase.	Ambiguous. The proportion of builds fixed by breaker increased when prototype 1 was installed, but then decreased to approximately baseline levels.
H8	The average duration of broken builds will decrease.	Supported. The proportion of builds fixed the day they were broken increased substantially when prototype 1 was installed, and then decreased somewhat but remained slightly above baseline levels for the rest of the study. Of the builds fixed the day they were broken, the mean fix duration decreased substantially and remained lower than the baseline, although did increase as the study progressed. Of the builds fixed on separate days, the mean fix duration was consistently lower than the baseline level.
H9	The total proportion of time in which any build plan is failing will decrease.	Not supported. While the proportion of time the USB light was red decreased substantially in prototype 1 relative to the baseline condition, the proportion increased dramatically in prototypes 2 and 3, well beyond baseline levels.

Two possible reasons for this type of effect are that the effect would have been consistent across all prototypes but quickly diminished as team members adjusted to the devices, or that the changes that were introduced as the study progressed – prototypes 2 and 3 – were less successful at impacting these metrics. Both of these possible explanations

appear plausible, and both have some degree of empirical support. Nevertheless the overall pattern of metrics, and the number of metrics exhibiting some degree of a Hawthorne effect, suggests that the initial effects of the system wore off as the devices became a part of the team's work environment. However, because of the additional alerts added in the second and third prototypes (namely, build failure notifications being given to all developers who committed to a failing plan, even if they committed working code to an already-failing plan), it became more difficult for each developer to establish exactly who was responsible for a given build failure and to then take individual responsibility for a given broken build. In some cases, breakages persisted for hours or even days, and as more developers committed to these plans over time the pool of potential breakers increased. This, in turn, diffused the responsibility over multiple team members, and reduced the usefulness of individually notifying potential breakers.

Despite the transient nature of some of these effects, it is also important to note that a number of effects that are indirectly associated with behaviours relating to a reduction in broken builds – such as H5 (builds manually triggered by a developer) and H6 (broken builds due to unit test failures) – did show a positive effect, and this effect was consistent for the entire study. Even though these effects were comparatively small and difficult to isolate, it would seem that some positive habits became ingrained.

A core principle upon which ambient devices are predicated is that an increase in team members' awareness into the build process will, either directly or indirectly, increases the entire team's overall sense of responsibility for the build process. This is thought not only to discourage broken builds, but also to encourage team members to quickly fix those builds which do break. The interplay between awareness and responsibility implies a measurable causal relationship between the introduction of a heightened level of awareness and a corresponding reduction in broken builds.

The metrics calculated for this study were not specifically designed to distinguish between the effects of awareness and responsibility. However, on consideration, it became apparent that the team's general awareness of build status would be represented through hypotheses H1 (that the proportion of broken builds would decrease) and, particularly, H8 (that the duration of broken builds would decrease as they were getting fixed more quickly). H1 had limited support from the data collected, but H8 did show a consistent decrease in the duration of broken build chains.

Similarly, the construct of individual responsibility can be inferred based on hypothesis H7 (the proportion of broken builds fixed by a breaker will increase). The results showed a Hawthorne effect for H7 that disappeared by the introduction of the second prototype.

B. Awareness and Responsibility

Overall, this pattern of results suggests that awareness of build failures does not necessarily lead to an increased sense of responsibility, even when one or a very few developers are individually alerted. Instead, these results indicate that a

distinction should be drawn between awareness and responsibility.

Awareness, in this context, is simply being made aware of the situation and of broken builds. This does seem to lead to an overall effect of fixing the broken builds more quickly as a team. Additionally, the simple fact that team members can easily scan the room for 'sad bunnies' means that they could become aware of broken build plans fairly easily, and with the added contextual information of knowing who was working on which components, could more easily work together to resolve these issues. These effects persisted across the study, were not affected by the specific prototype behaviours, and were also reflected in the comments made by team members.

Individual responsibility for fixing broken builds is more complex. The results suggest that the introduction of the devices had mixed effects on team members' sense of responsibility. While developers did initially fix their own broken builds more often, this effect was eliminated by the time the second prototype was installed. As noted above, the behavioural characteristics of the second and third prototypes did mean that more developers were potentially given notifications for any given broken build.

Despite these complexities, and the limited quantitative support for the idea that individual responsibility was increased by these devices, a number of team members reported that they did feel an increased sense of responsibility when they caused a broken build, largely because they were cognisant of their red rabbit advertising this fact to the team at large. However, they also noted that they were not always able to fix these problems immediately due to the complexity of the task or factors beyond their control.

Accordingly, it is important to distinguish between two types of individual responsibility. An external (or exogenous) sense of responsibility is due largely to peer pressure applied through mechanisms such as friendly teasing or commenting about a 'sad bunny'. This form of responsibility wore off as the novelty wore off. An internal (or endogenous) sense of responsibility, as indicated by the feeling of responsibility or embarrassment, appeared consistent throughout the study, based on statements from team members, but did not seem to have a noticeable impact on the quantitative metrics. Further study would be needed to isolate these types of responsibility and determine which interventions might result in lasting increases in either or both.

The most striking example of a behavioural change was the developer who, having previously been known for ignoring failing builds, became enthusiastic about the project and later commented that he was now actually paying attention to builds. Whether this change resulted in a measurable difference in the developer's work performance was not something that could be ascertained from our data. However, because software development is a team exercise, and given the other team members' stance that broken builds disrupt the development process and frustrate other team members, it is reasonable to expect that their opinions of this

developer's work would have improved given the change in his behaviour.

C. Productivity

We observed a higher number of builds, but also an increasing number of files per changeset. This pattern of results was somewhat unexpected; our initial expectation was that an increasing frequency of builds would lead to a corresponding decrease in the size of each build's changeset. Three possible reasons for this incongruity are: the way the changeset size metric was calculated; more code was being written; or there was an unanticipated effect of the intervention. The most plausible is the calculation of the changeset size metric as it only considered the number of source code files added or changed, and did not consider the size of each code file change. Alternatively more code may have actually been written and committed. Such a difference could be considered an increase in productivity: developers wrote more code and did so more quickly. It is also a theoretical possibility that such a difference could be due to the interventions that were performed.

The probability that an increase in productivity is directly and exclusively due to the introduction of ambient build monitoring devices is low. This theory has minimal prior plausibility and little empirical justification beyond the correlation between these two metrics. However, it does serve to emphasise the point that the effects of the build process can be widespread within a team, and that little is known about the effects of the build process on other aspects of team-based software development. Other metrics, such as the rate at which work items are completed, may be more accurate measures of this construct.

D. Ambient Devices

An important aspect of this project was the evaluation of the individual ambient devices, in addition to a shared device that other projects have used (e.g. [12]). Having the two device types proved to be invaluable, with team members holding very different opinions and impressions of each device type. The individual Nabaztag rabbits were extremely popular, even after several months. Every team member commented that they had positive opinions of the rabbits, considered them fun, and that they suited the personalities of the team. The term 'sad bunny' appeared to have entered the team's vernacular and had developed into a euphemistic synonym for 'broken build'.

The central light provoked more varied opinions. Multiple build plans were aggregated on a single light, and this level of aggregation was too high to be useful. When evaluating hypothesis H9 (the time the central light showed red vs. green) it became apparent that the proportion of time that the central light showed red was increasing dramatically over the study. This is because of the increased number of builds (H2) and proportion broken builds (H1) meant that broken build chains were frequently overlapping. Thus the central light did not provide adequate information on the status of individual builds. This highlights the importance of taking a team's workflow into account when designing these types of interventions. Despite this caveat, some team

members did find the central light valuable saying they used it to get a general impression of the team's status as they were walking in and out of the open plan work area.

A core tenet of ambient devices is that information is presented through a distinct channel. At the commencement of the study some developers argued that the system should run on the participants' workstations. However, as the study proceeded the bunnies became synonymous with the build process, as was evident from their comments about 'sad bunnies'. In addition, information presented through ambient devices should be immediately discernible. This required us to distil the complex build information into states. We then encoded this on the bunnies as 'happy' or 'sad' bunnies via ear position and LED light colours. There is scope to encode more than two states on an ambient device and for them to still retain their essential features of immediate cognition and passivity. Exploring more states was one suggestion from the team that is yet to be investigated.

In summary, the results from this study indicate that awareness and responsibility are not as tightly linked as we initially expected. The team's overall awareness of broken builds was certainly increased, based on the developers' comments and on the consistently lower duration of broken build chains. However, the degree of individual responsibility for fixing these broken build chains is substantially more difficult to isolate. The results suggested that while developers did feel a consistently stronger sense of internal responsibility from having these devices, this did not translate into any long-term measurable effects. External responsibility, due to teammates' commenting or teasing, was demonstrated but did not persist in the long term. Accordingly, we can conclude that while individual responsibility for broken build fixes does require awareness of the build status, this awareness does not necessarily lead to responsibility.

VII. IMPLICATIONS AND ADVICE TO PRACTITIONERS

Because of the extent to which these devices are beginning to be used within industry, it is important to consider the implications of these findings for other teams. [24] presents overall guidelines for the design of ambient visualisations in the workplace. This section extends their work by providing general advice for practitioners who may wish to implement ambient build monitoring systems in other organisations.

First, it is of critical importance that any devices intended to support ambient awareness fit within the team's practices and workflow. Different teams will emphasise different aspects of their development methodology, of the information they generate and obtain, and of their practices and processes. In this case study team a great deal of emphasis was placed on the build process, but this was tempered by a level of practicality and an acknowledgment that broken builds could not always be avoided.

Second, the nature of the devices needs a great deal of consideration. While many types of displays and devices can present what is fundamentally the same information, the subtle differences between each type of device can have profound effects on the perceptions of that device and of the

team's willingness to integrate it into their work. In this case, the form factor of the Nabaztag rabbits was very popular within the team; devices with similar overall output but a different form factor, such as a simple LED, would probably not have engendered such a positive response, and accordingly would likely not have been accepted by team members as readily.

Third, consideration must be given to whether a single central device or a set of individual devices is most appropriate. This decision will be based on factors including the nature of the projects the team are working on, the size of the team, and the available resources. In this case study individual devices were more appropriate and useful due to the large number of developers and the multiple build plans. Aggregating these multiple data points onto the central device necessarily involved a loss of information, and accordingly diluted the value of the system too much. A smaller team, working on a single codebase, may not have this same problem.

Fourth, strong consideration must be given to the simplicity and glanceability of the information presented on any such device. An ambient device will only be successful if the information it presents is easily perceivable. When dealing with general-purpose devices such as Nabaztag rabbits, there is a strong inclination to integrate multiple channels of information and to use the full set of device features to provide as much information as possible. However, such a practice will likely make the device difficult to comprehend. In this case study, even the relatively simple behaviour of the devices was occasionally misinterpreted.

Finally, objective criteria should be established for measuring the success and impact of such a system. The hypotheses listed in Table 3 provide a useful starting point. Ultimately, the designer of such a system should be prepared to make alterations to its behaviour depending on the results of such an evaluation. Some effects will likely be short-term in nature due to the Hawthorne effect. A focus on long-term quality and performance improvements is likely to require a long-term trial.

VIII. CONCLUSIONS AND FUTURE WORK

We have described an ambient system for build status notification and its evaluation in use in an industrial setting with an agile development team. Team members reported that the ambient devices gave them an increased sense of awareness of, and responsibility for, broken builds. Additionally, some individual developers noted that the system dramatically changed their perception of the build process and made them more cognisant of broken builds. Despite these comments, there was no measurable change in the proportion of builds that broke following the introduction of the ambient devices. However, and significantly, the overall number of builds increased substantially, and the duration of broken builds decreased indicating enhanced efficiency of the software development process. Additionally, the properties of the ambient devices – separation of channel, simplicity, glanceability, and

redundancy of encoding – were validated as core components of a successful ambient awareness system.

This work has suggested several potential future directions for research into the area of ambient build status monitoring.

Undertaking additional case studies using the framework we have developed is an obvious first step. This would allow generalization of results and more definitive conclusions on the efficacy of the approaches we have described. Another option is a longitudinal study over an extended period to provide more definitive information on longer-term trends, and the inclusion of return-to-baseline conditions to mitigate the Hawthorne effect.

In some cases the build server data was inadequate to permit accurate notifications. This could be improved by including data from other sources, such as an issue management system, to more accurately identify, for example, build break responsibility. Another alternative would be to use a build server with more complete build information.

We have analysed the use of two ambient devices in this work. There are, as discussed earlier, a range of different ambient build awareness devices. Investigating other devices or combinations of devices would be worthwhile.

Finally, the extent to which ambient devices can be viewed strictly as sources of build failure notifications versus providing reinforcement and punishment is an area that requires further study. A wide literature exists on the effects of reinforcement and punishment, and suggests that pairing reinforcement and punishment together will have the greatest effect. This question has implications broader than purely within build notifications, with ambient devices becoming more common in myriad other application areas including homes, hospitals, offices, and even on roads. As ambient devices become seen as a legitimate option for presenting complex and dynamic information in a contextually appropriate and cognitively minimalistic manner, the exploration of their meanings and properties, and of the design characteristics associated with their success, will gain increasing importance.

ACKNOWLEDGMENT

We wish to express our appreciation to our participants, and their employer, for giving us their time and expertise. We also wish to acknowledge the New Zealand Ministry for Science and Innovation via the Software Process and Product Improvement project for providing financial support.

REFERENCES

- [1] D. Saff and M. D. Ernst, "An experimental evaluation of continuous testing during development," in *Proc. ACM SIGSOFT 2004*, Boston, MA, 2004, pp. 76-85.
- [2] M. Fowler, "Continuous integration," 2006. [Online]. Available: <http://www.martinfowler.com/articles/continuousIntegration.html>. [Accessed: 30-Jan-2012].
- [3] J. Holck and N. Jorgensen, "Continuous integration and quality assurance: A case study of two open source projects," *AJIS*, vol. 11, pp. 40-53, 2004.

- [4] D. Damian, L. Izquierdo, J. Singer, and I. Kwan, "Awareness in the wild: Why communication breakdowns occur," in *Proc. ICGSE 2007*, Munich, Germany, 2007, pp. 81-90.
- [5] J. Downs, J. Hosking, and B. Plimmer, "Status communication in agile software teams: A case study," in *Proc. ICSEA 2010*, Nice, France, 2010, pp. 82-87.
- [6] C. Gutwin, R. Penner, and K. Schneider, "Group awareness in distributed software development," in *Proc. CSCW 2004*, Chicago, IL, 2004, pp. 72-81.
- [7] J. T. Biehl, M. Czerwinski, G. Smith, and G. G. Robertson, "FASTDash: A visual dashboard for fostering awareness in software teams," in *Proc. CHI 2007*, San Jose, CA, 2007, pp. 1313-1322.
- [8] A. Sarma, Z. Noroozi, and A. van der Hoek, "Palantir: Raising awareness among configuration management workspaces," in *Proc. ICSE 2003*, Portland, Oregon, 2003, pp. 444-454.
- [9] M. Clark, *Pragmatic Project Automation*. Raleigh, NC: The Pragmatic Programmers, LLC, 2004.
- [10] M. Swanson, "Automated continuous integration and the Ambient Orb," 2004. [Online]. Available: <http://blogs.msdn.com/mswanson/articles/169058.aspx>. [Accessed: 30-Jan-2012].
- [11] M. Woodward, "Team Build 2008 API Example: The Build Wallboard," 2007. [Online]. Available: <http://www.woodwardweb.com/vsts/000395.html>. [Accessed: 30-Jan-2012].
- [12] M. Woodward, "Brian the Build Bunny," 2008. [Online]. Available: <http://www.woodwardweb.com/gadgets/000434.html>. [Accessed: 30-Jan-2012].
- [13] J.-L. de Morlhon, "Nabaztag Scala Library," 2009. [Online]. Available: <http://morlhon.net/blog/2009/11/09/nabaztag-scala-library/>. [Accessed: 30-Jan-2012].
- [14] J. Atwood, "Automated Continuous Integration and the BetaBrite LED Sign," 2005. [Online]. Available: <http://www.codinghorror.com/blog/archives/000238.html>. [Accessed: 30-Jan-2012].
- [15] S. Harrison, "Putting the SnowMan to work: Cruise Control .Net Build Status Monitor," 2007. [Online]. Available: <http://blog.analysisuk.com/post/33-Putting-the-SnowMan-to-work-Cruise-Control-Net-Build-Status-Monitor.aspx>. [Accessed: 30-Jan-2012].
- [16] R. Ablett, F. Maurer, E. Sharlin, J. Denzinger, and C. Schock, "Build notifications in agile environments," in *Agile Processes in Software Engineering and Extreme Programming*, vol. 9, P. Abrahamsson, R. Baskerville, K. Conboy, B. Fitzgerald, L. Morgan, and X. Wang, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 230-231.
- [17] E. L. Thorndike, *Animal Intelligence*. New York: Macmillan, 1911.
- [18] Atlassian Pty Ltd, "Bamboo Remote API," 2009. [Online]. Available: <http://confluence.atlassian.com/display/BAMBOO022/Bamboo+Remote+API>. [Accessed: 30-Jan-2012].
- [19] University of Tampere, "jNabServer - Control your bunny," 2008. [Online]. Available: <http://www.cs.uta.fi/hci/spi/jnabserver/>. [Accessed: 30-Jan-2012].
- [20] D. H. Barlow and M. Hersen, *Single Case Experimental Designs: Strategies for Studying Behavior Change*. New York: Pergamon Press Inc., 1984.
- [21] J. D. Elashoff and C. E. Thoresen, "Choosing a statistical method for analysis of an intensive experiment," in *Single Subject Research: Strategies for Evaluating Change*, New York: Academic Press, 1978.
- [22] R. E. Clark and B. M. Sugrue, "Research on instructional media, 1978-1988," *Instructional Technology: Past, Present and Future*, pp. 336-346, 1996.
- [23] E. Mayo, *Hawthorne and the Western Electric Company: The Social Problems of an Industrial Civilisation*. Routledge, 1949.
- [24] C. Parnin and C. Gorg, "Design guidelines for ambient software visualization in the workplace," in *Proceedings of the 4th IEEE International Workshop on Visualizing Software for Understanding and Analysis, 2007. VISSOFT 2007*, 2007, pp. 18-25.